

## **Técnicas de pruebas de software**

### **1. Introducción: ¿qué son las pruebas de software?**

Las pruebas de software son un conjunto de actividades planificadas que permiten verificar y validar que un sistema, aplicación, módulo o componente funcione correctamente, cumpla con los requisitos definidos y entregue valor al usuario final.

En términos simples, probar software significa revisar si el sistema hace lo que debe hacer, si lo hace de forma correcta, si responde adecuadamente ante errores, si es seguro, rápido, usable, mantenible y confiable.

Las pruebas no sirven solo para encontrar errores. También permiten prevenir fallas, reducir riesgos, mejorar la calidad del producto y entregar mayor confianza antes de liberar una versión al usuario o cliente.

Según el enfoque ISTQB, las técnicas de prueba se agrupan principalmente en técnicas de caja negra, caja blanca, basadas en experiencia y enfoques colaborativos. Estas permiten diseñar casos de prueba desde distintas perspectivas: requisitos, estructura interna del código, experiencia del tester o colaboración con usuarios y desarrolladores.

---

### **2. Importancia de las pruebas de software**

Las pruebas son importantes porque permiten:

Detectar defectos antes de que el software llegue al usuario final.

Reducir costos, ya que corregir errores en producción suele ser mucho más caro que corregirlos durante el desarrollo.

Aumentar la confiabilidad del sistema.

Validar que los requisitos funcionales y no funcionales se cumplan.

Mejorar la seguridad, rendimiento, usabilidad y mantenibilidad.

Apoyar la toma de decisiones antes de liberar una versión.

Disminuir riesgos técnicos, económicos, operacionales y reputacionales.

En proyectos reales, un sistema puede funcionar “aparentemente bien”, pero fallar ante muchos usuarios conectados, datos incorrectos, ataques, integraciones externas, cambios de configuración o escenarios no previstos. Por eso, las pruebas deben ser parte permanente del ciclo de vida del software.

---

### **3. Rol de las pruebas dentro de las metodologías de software**

#### **En metodologías tradicionales**

En modelos como cascada, las pruebas suelen aparecer después del análisis, diseño, codificación e implementación. Primero se construye el sistema y luego se valida.

Este enfoque permite una planificación ordenada, pero tiene una debilidad importante: si los errores se detectan tarde, corregirlos puede ser costoso.

### **En metodologías ágiles**

En metodologías ágiles como Scrum o Kanban, las pruebas se integran durante todo el desarrollo. Cada incremento, historia de usuario o sprint debe ser probado de forma continua.

Aquí las pruebas cumplen un rol clave porque permiten entregar versiones funcionales frecuentes. Se usan pruebas unitarias, integración continua, pruebas automatizadas, revisión de criterios de aceptación y pruebas exploratorias.

### **En DevOps y DevSecOps**

En DevOps, las pruebas se integran al pipeline de integración y despliegue continuo. Cada cambio en el código puede activar pruebas automáticas.

En DevSecOps, además, se incorporan pruebas de seguridad desde etapas tempranas, como SAST, DAST y SCA, para detectar vulnerabilidades antes de producción. La guía australiana de desarrollo seguro recomienda incluir SAST, DAST y SCA para lograr una cobertura más completa de seguridad.

---

## **4. Clasificación general de las pruebas de software**

Las pruebas pueden organizarse de varias formas:

- Por conocimiento interno del sistema: caja negra, caja blanca y caja gris.
- Por nivel: unitarias, integración, sistema y aceptación.
- Por objetivo: funcionales y no funcionales.
- Por ejecución: manuales o automatizadas.
- Por momento: estáticas o dinámicas.
- Por propósito específico: regresión, humo, sanidad, seguridad, rendimiento, usabilidad, compatibilidad, accesibilidad, entre otras.

---

## **5. Técnicas de pruebas según el conocimiento del sistema**

### **5.1 Pruebas de caja negra**

Las pruebas de caja negra evalúan el sistema desde fuera, sin revisar el código fuente. El tester se enfoca en entradas, salidas, reglas de negocio y comportamiento esperado.

Son muy utilizadas para validar requisitos funcionales.

Técnicas principales de caja negra:

Partición de equivalencia: divide los datos de entrada en grupos válidos e inválidos. Por ejemplo, si una edad válida está entre 18 y 65 años, se prueban valores dentro y fuera de ese rango.

Análisis de valores límite: prueba los bordes de los rangos, porque allí suelen aparecer errores. Por ejemplo: 17, 18, 65 y 66.

Tablas de decisión: se usan cuando el sistema depende de varias condiciones combinadas.

Pruebas de transición de estados: se aplican cuando el sistema cambia de estado. Por ejemplo, pedido creado, pagado, enviado y cancelado.

Pruebas de casos de uso: validan escenarios completos desde la perspectiva del usuario.

ISTQB identifica como técnicas comunes de caja negra la partición de equivalencia, análisis de valores límite, tablas de decisión y transición de estados.

Herramientas posibles:

- Selenium: automatización de pruebas web.
- Cypress: pruebas funcionales y end-to-end para aplicaciones web.
- Playwright: automatización moderna para pruebas web en varios navegadores.
- TestRail: gestión de casos de prueba.

---

## 5.2 Pruebas de caja blanca

Las pruebas de caja blanca analizan la estructura interna del código. Aquí sí se conoce la lógica interna, los caminos de ejecución, condiciones, ciclos, funciones y ramas.

Son frecuentes en pruebas unitarias y análisis técnico.

Técnicas principales:

- Cobertura de sentencias: verifica que se ejecuten las líneas de código.
- Cobertura de ramas: prueba las decisiones, como if, else, switch.
- Cobertura de condiciones: analiza combinaciones lógicas.
- Pruebas de caminos: revisa distintas rutas posibles del programa.
- Pruebas de ciclos: valida bucles con cero, una, varias y muchas iteraciones.

Herramientas posibles:

- JUnit: pruebas unitarias en Java.
  - pytest: pruebas unitarias y funcionales en Python.
  - NUnit: pruebas para .NET.
  - JaCoCo: medición de cobertura en Java.
  - Coverage.py: cobertura de código en Python.
-

### 5.3 Pruebas de caja gris

Las pruebas de caja gris combinan elementos de caja negra y caja blanca. El tester no necesariamente conoce todo el código, pero sí tiene información parcial sobre arquitectura, base de datos, APIs, flujos internos o diseño técnico.

Son útiles para probar integraciones, APIs, seguridad y flujos complejos.

Herramientas posibles:

- Postman: pruebas de APIs.
  - SoapUI: pruebas de servicios web SOAP y REST.
  - Burp Suite: pruebas de seguridad web.
  - Insomnia: pruebas de APIs REST y GraphQL.
- 

## 6. Pruebas según el nivel del software

### 6.1 Pruebas unitarias

Validan pequeñas unidades de código, como funciones, métodos, clases o componentes individuales. Su objetivo es comprobar que cada parte mínima funcione correctamente de manera aislada.

Ejemplo: probar que una función que calcula el IVA retorne el monto correcto.

Son rápidas, automatizables y muy importantes en desarrollo ágil. Herramientas como Jest, PyTest, JUnit, Vitest y NUnit siguen siendo ampliamente utilizadas en pruebas unitarias.

Herramientas:

- JUnit: Java.
  - PyTest: Python.
  - Jest: JavaScript.
  - Vitest: JavaScript/TypeScript.
  - NUnit: C#/.NET.
- 

### 6.2 Pruebas de integración

Validan que dos o más módulos funcionen correctamente al interactuar.

Ejemplo: verificar que el módulo de ventas se comunique correctamente con inventario y facturación.

Tipos:

- Integración incremental.
- Integración big bang.
- Integración descendente.

- Integración ascendente.
- Integración por contratos.

Herramientas:

- Postman: APIs.
  - SoapUI: servicios web.
  - Testcontainers: pruebas con contenedores reales.
  - REST Assured: APIs REST en Java.
  - WireMock: simulación de servicios externos.
- 

### **6.3 Pruebas de sistema**

Evalúan el sistema completo en un entorno similar al real. Su objetivo es verificar que todos los componentes funcionen como una solución integrada.

Ejemplo: probar una plataforma de matrícula completa, desde el inicio de sesión hasta el pago y generación de comprobante.

Herramientas:

- Selenium.
  - Playwright.
  - Cypress.
  - Robot Framework.
  - Katalon Studio.
- 

### **6.4 Pruebas de aceptación**

Verifican si el sistema cumple con las necesidades del cliente, usuario o negocio. Se relacionan directamente con los criterios de aceptación.

Tipos:

- UAT, pruebas de aceptación de usuario.
  - Pruebas alfa.
  - Pruebas beta.
  - Pruebas de aceptación operacional.
  - Herramientas:
    - Cucumber: pruebas BDD con lenguaje Gherkin.
    - SpecFlow: BDD para .NET.
    - FitNesse: pruebas de aceptación colaborativas.
    - TestRail: gestión de pruebas de aceptación.
-

## 7. Pruebas funcionales

Las pruebas funcionales verifican que el sistema haga lo que debe hacer según los requisitos. Se enfocan en funciones, reglas de negocio, validaciones, entradas, salidas y flujos.

Ejemplo: comprobar que un usuario pueda registrarse, iniciar sesión, recuperar contraseña y actualizar sus datos.

Tipos principales:

- Pruebas de interfaz.
- Pruebas de APIs.
- Pruebas de base de datos.
- Pruebas end-to-end.
- Pruebas de regresión funcional.

Herramientas:

- Selenium.
  - Cypress.
  - Playwright.
  - Postman.
  - Robot Framework.
- 

## 8. Pruebas no funcionales

Las pruebas no funcionales evalúan cómo se comporta el sistema. No revisan solo si una función existe, sino su calidad.

Incluyen rendimiento, seguridad, usabilidad, accesibilidad, compatibilidad, escalabilidad, confiabilidad, mantenibilidad y portabilidad.

---

### 8.1 Pruebas de rendimiento

Evalúan velocidad, estabilidad, consumo de recursos y capacidad de respuesta.

Tipos:

- Pruebas de carga: comportamiento con usuarios esperados.
- Pruebas de estrés: comportamiento bajo condiciones extremas.
- Pruebas de volumen: grandes cantidades de datos.
- Pruebas de resistencia: funcionamiento durante largo tiempo.
- Pruebas de escalabilidad: capacidad de crecer ante mayor demanda.

Herramientas:

- Apache JMeter.

- Gatling.
- k6.
- Locust.
- LoadRunner.

---

## 8.2 Pruebas de seguridad

Buscan vulnerabilidades, configuraciones inseguras y riesgos de ataque.

Incluyen:

- SAST: análisis estático del código fuente.
- DAST: análisis dinámico sobre la aplicación en ejecución.
- SCA: análisis de dependencias y librerías.
- IAST: análisis interactivo durante la ejecución.
- Pentesting: pruebas manuales o semiautomatizadas de intrusión.

GitLab explica que SAST permite descubrir vulnerabilidades en el código antes de llegar a producción, integrado en pipelines CI/CD. OWASP también reconoce ZAP como una herramienta DAST gratuita y de código abierto para escaneo de vulnerabilidades y apoyo en pruebas manuales web.

Herramientas:

- OWASP ZAP: DAST.
- Burp Suite: análisis de seguridad web.
- SonarQube: análisis estático y calidad de código.
- Snyk: vulnerabilidades en dependencias y contenedores.
- GitLab SAST: análisis de seguridad integrado en CI/CD.
- Semgrep: análisis estático flexible.

---

## 8.3 Pruebas de usabilidad

Evalúan si el sistema es fácil de usar, comprensible, eficiente y satisfactorio para el usuario.

Aspectos evaluados:

- Claridad de navegación.
- Facilidad de aprendizaje.
- Cantidad de errores cometidos por usuarios.
- Tiempo necesario para completar tareas.
- Satisfacción del usuario.

Herramientas:

- Maze.

- UserTesting.
  - Lookback.
  - Hotjar.
  - Optimal Workshop.
- 

#### **8.4 Pruebas de accesibilidad**

Verifican que el software pueda ser usado por personas con distintas capacidades, por ejemplo usuarios con discapacidad visual, motora, auditiva o cognitiva.

Se basan en criterios como WCAG.

Herramientas:

- axe DevTools.
  - WAVE.
  - Lighthouse.
  - Pa11y.
  - NVDA, lector de pantalla.
- 

#### **8.5 Pruebas de compatibilidad**

Validan que el sistema funcione correctamente en distintos navegadores, sistemas operativos, dispositivos, resoluciones o versiones.

Ejemplo: probar una aplicación web en Chrome, Firefox, Edge, Safari, Android e iOS.

Herramientas:

- BrowserStack.
  - LambdaTest.
  - Sauce Labs.
  - CrossBrowserTesting.
  - Playwright.
- 

#### **8.6 Pruebas de confiabilidad**

Evalúan si el sistema puede operar sin fallas durante un periodo determinado.

Incluyen:

- Tolerancia a fallos.
- Recuperación ante errores.
- Disponibilidad.
- Continuidad operativa.



Herramientas:

- Chaos Monkey.
- Gremlin.
- LitmusChaos.
- Prometheus.
- Grafana.

---

## 9. Pruebas según el momento de ejecución

### 9.1 Pruebas estáticas

Se realizan sin ejecutar el programa. Analizan código, documentación, requisitos, diseño o arquitectura.

Ejemplos:

- Revisión de código.
- Inspección de documentos.
- Análisis estático.
- Revisión de estándares.
- Detección de malas prácticas.

Herramientas:

- SonarQube.
- ESLint.
- Pylint.
- Checkstyle.
- Semgrep.

---

### 9.2 Pruebas dinámicas

Se realizan ejecutando el software. Permiten observar el comportamiento real del sistema.

Ejemplos:

- Pruebas funcionales.
- Pruebas de integración.
- Pruebas de rendimiento.
- Pruebas de seguridad dinámica.

Herramientas:

- Selenium.
- Cypress.
- Postman.

- JMeter.
  - OWASP ZAP.
- 

## **10. Pruebas manuales y automatizadas**

### **10.1 Pruebas manuales**

Son ejecutadas por una persona sin scripts automatizados. Son útiles cuando se requiere criterio humano, exploración, evaluación visual o validación de experiencia de usuario.

Ventajas:

- Flexibilidad.
- Útiles en etapas tempranas.
- Permiten descubrir errores inesperados.
- Buenas para usabilidad y exploración.

Desventajas:

- Más lentas.
- Más propensas a errores humanos.
- Difíciles de repetir muchas veces.

Herramientas:

- TestRail.
  - Zephyr.
  - Xray.
  - qTest.
  - Jira.
- 

### **10.2 Pruebas automatizadas**

Se ejecutan mediante scripts o herramientas. Son ideales para pruebas repetitivas, regresión, integración continua y validación frecuente.

Ventajas:

- Rapidez.
- Repetibilidad.
- Integración con CI/CD.
- Menor esfuerzo en regresiones.

Desventajas:

- Requieren mantenimiento.
- No todo debe automatizarse.

- Pueden ser costosas al inicio.

Herramientas:

- Selenium.
  - Cypress.
  - Playwright.
  - JUnit.
  - PyTest.
  - Jenkins.
  - GitHub Actions.
  - GitLab CI/CD.
- 

## **11. Pruebas específicas del ciclo de desarrollo**

### **11.1 Pruebas de humo**

Verifican rápidamente que las funciones principales del sistema estén operativas después de una compilación o despliegue.

Ejemplo: validar que la aplicación abre, permite iniciar sesión y carga la pantalla principal.

Herramientas:

- Selenium.
  - Cypress.
  - Playwright.
  - Postman.
  - Robot Framework.
- 

### **11.2 Pruebas de sanidad**

Validan una funcionalidad específica después de un cambio menor o corrección.

Ejemplo: si se corrigió el módulo de cambio de contraseña, se prueba principalmente esa funcionalidad.

Herramientas:

- Postman.
  - Cypress.
  - Selenium.
  - TestRail.
  - Zephyr.
-

### 11.3 Pruebas de regresión

Comprueban que nuevas modificaciones no hayan dañado funcionalidades existentes.

Son fundamentales en proyectos ágiles y DevOps.

Herramientas:

- Selenium.
  - Cypress.
  - Playwright.
  - JUnit.
  - PyTest.
  - Robot Framework.
- 

### 11.4 Pruebas exploratorias

El tester explora el sistema usando su experiencia, intuición y conocimiento del dominio. No sigue necesariamente casos de prueba rígidos.

Son útiles para encontrar defectos no previstos por los casos formales.

Herramientas:

- TestRail.
  - Xray.
  - Session Tester.
  - qTest.
  - Jira.
- 

## 12. Pruebas de API

Validan servicios REST, SOAP, GraphQL o microservicios. Revisan endpoints, códigos de respuesta, autenticación, datos enviados y recibidos, errores y rendimiento.

Ejemplo: verificar que POST /usuarios cree correctamente un usuario y retorne código 201.

Herramientas:

- Postman.
  - Insomnia.
  - SoapUI.
  - REST Assured.
  - Karate DSL.
- 

## 13. Pruebas de base de datos

Validan integridad, consultas, procedimientos almacenados, triggers, restricciones, relaciones y consistencia de datos.

Ejemplo: comprobar que al eliminar un cliente no queden ventas huérfanas.

Herramientas:

- SQLUnit.
  - DbUnit.
  - tSQLt.
  - pgTAP.
  - DBeaver.
- 

## **14. Pruebas end-to-end**

Simulan flujos completos del usuario desde el inicio hasta el fin.

Ejemplo: registrarse, iniciar sesión, seleccionar producto, pagar y recibir confirmación.

Atlassian señala que las pruebas end-to-end replican el comportamiento del usuario en un entorno completo, aunque suelen ser más costosas y difíciles de mantener que las pruebas de menor nivel.

Herramientas:

- Cypress.
  - Playwright.
  - Selenium.
  - TestCafe.
  - Katalon Studio.
- 

## **15. Pruebas móviles**

Evalúan aplicaciones Android e iOS considerando dispositivos, tamaños de pantalla, sensores, permisos, conectividad, batería y rendimiento.

Herramientas:

- Appium.
  - Espresso.
  - XCUITest.
  - Firebase Test Lab.
  - BrowserStack App Automate.
- 

## **16. Pruebas de instalación y configuración**

Validan que el software pueda instalarse, actualizarse, configurarse y desinstalarse correctamente.

Ejemplo: probar instalación en Windows, Linux y macOS.

Herramientas:

- InstallShield.
  - Advanced Installer.
  - Docker.
  - Ansible.
  - Chef.
- 

## **17. Pruebas de recuperación**

Evalúan cómo responde el sistema ante fallos, caídas, interrupciones, pérdida de red o errores de infraestructura.

Herramientas:

- Gremlin.
  - Chaos Monkey.
  - LitmusChaos.
  - Kubernetes probes.
  - Prometheus + Grafana.
- 

## **18. Pruebas de mantenibilidad**

Evalúan qué tan fácil es modificar, extender o corregir el software.

Aspectos:

- Complejidad del código.
- Duplicación.
- Cobertura de pruebas.
- Deuda técnica.
- Calidad estructural.

Herramientas:

- SonarQube.
  - CodeClimate.
  - Codacy.
  - ESLint.
  - Pylint.
-

## 19. Pruebas de localización e internacionalización

Verifican que el software funcione correctamente en distintos idiomas, regiones, monedas, formatos de fecha, zonas horarias y convenciones culturales.

Herramientas:

- Lokalise.
- Crowdin.
- Phrase.
- Selenium.
- Playwright.

---

## 20. Tabla resumen de tipos de pruebas y herramientas

| Tipo de prueba | Propósito principal                       | Herramientas                          |
|----------------|-------------------------------------------|---------------------------------------|
| Unitarias      | Probar funciones o métodos aislados       | JUnit, pytest, Jest                   |
| Integración    | Validar comunicación entre módulos        | Postman, Testcontainers, REST Assured |
| Sistema        | Evaluar el sistema completo               | Selenium, Playwright, Robot Framework |
| Aceptación     | Validar requisitos del usuario            | Cucumber, SpecFlow, FitNesse          |
| Funcionales    | Verificar comportamiento esperado         | Cypress, Selenium, Postman            |
| Rendimiento    | Medir carga, estrés y velocidad           | JMeter, Gatling, k6                   |
| Seguridad      | Detectar vulnerabilidades                 | OWASP ZAP, Burp Suite, SonarQube      |
| Usabilidad     | Evaluar facilidad de uso                  | Maze, UserTesting, Hotjar             |
| Accesibilidad  | Validar uso inclusivo                     | axe DevTools, WAVE, Lighthouse        |
| Compatibilidad | Probar navegadores/dispositivos           | BrowserStack, LambdaTest, Sauce Labs  |
| Regresión      | Verificar que cambios no rompan funciones | Selenium, Cypress, Playwright         |
| API            | Validar servicios y endpoints             | Postman, Insomnia, SoapUI             |
| Base de datos  | Probar datos, consultas y restricciones   | DbUnit, tSQLt, pgTAP                  |
| Móvil          | Probar apps Android/iOS                   | Appium, Espresso, XCUITest            |
| Exploratorias  | Descubrir errores no previstos            | TestRail, Xray, Jira                  |
| Estáticas      | Analizar sin ejecutar código              | SonarQube, ESLint, Pylint             |
| Dinámicas      | Evaluar con software ejecutándose         | Selenium, JMeter, OWASP ZAP           |

---

## 21. Conclusión

Las técnicas de pruebas de software son esenciales para asegurar la calidad de un sistema. No existe una única técnica suficiente para todos los escenarios. Un proyecto sólido combina pruebas unitarias, integración, sistema, aceptación, regresión, seguridad, rendimiento, usabilidad y accesibilidad.

En metodologías modernas, las pruebas no son una etapa final, sino una actividad continua. En enfoques ágiles, DevOps y DevSecOps, probar se convierte en una práctica permanente que acompaña al desarrollo, automatiza validaciones, reduce riesgos y mejora la confianza en cada entrega.

La elección de herramientas depende del lenguaje, tipo de aplicación, arquitectura, presupuesto, experiencia del equipo y nivel de automatización requerido. Lo importante es que la estrategia de pruebas esté alineada con los riesgos del proyecto, los requisitos del negocio y las necesidades reales de los usuarios.